

aeacrA

基于 raylib 下落式节奏游戏

A falling rhythm game built with raylib

项目概览 / Project Overview

游戏类型：6 轨下落式节奏游戏（类 Arcaea）

Game Type: 6-lane falling rhythm game (Arcaea-style)

渲染引擎：raylib 5.5

Rendering Engine: raylib 5.5

开发环境：macOS + Clang++ 17 + VS Code + Makefile

Development Environment: macOS + Clang++ 17 + VS Code + Makefile

代码规模：14 个头文件，11 个源文件

Code Scale: 14 headers, 11 source files

外部依赖：raylib、aubio（音频特征分析）

Dependencies: raylib, aubio (audio feature analysis)

按键布局 / **Key Layout:** S D F J K L — 左手无名指 → 右手无名指，对称设计

Left ring → right ring, symmetric ergonomic layout

项目结构 / Project Structure

```
aeacrA/
├── include/           # 头文件
│   ├── Config.h      # 全局常量
│   ├── Chart.h        # 谱面
│   ├── ChartGenerator.h # 谱面生成
│   ├── JsonParser.h   # JSON 解析
│   ├── Judge.h        # 判定
│   ├── InputManager.h # 输入管理器
│   ├── AudioManager.h # 音频播放
│   ├── ParticleSystem.h # 粒子特效
│   ├── Renderer.h     # 透视渲染器
│   ├── SongManager.h  # 歌曲库管理
│   ├── SongEntry.h    # 歌曲数据条目
│   ├── FilePicker.h   # 文件选择器接口
│   └── Game.h         # 游戏
├── src/              # 实现
├── charts/           # 手写谱面
└── songs/            # 导入歌曲
```

架构 / Architecture

- `Chart` — 谱面数据 / Chart data (Note list, JSON serialization)
- `JudgeSystem` — 判定 / Judgment (PERFECT / GREAT / GOOD / MISS)
- `Renderer` — 渲染 / Rendering (perspective, HUD, menus)
- `AudioManager` — 音乐流播放 / Music stream playback & beat pulse
- `InputManager` — 按键事件 / Key press/release events
- `ParticleSystem` — 粒子特效 / Particle effects (explosion/spiral/fire)
- `SongManager` — 歌曲管理 / Song scanning, import, chart generation
- `FilePicker` — 文件选择器 / Native file picker (Obj-C++ bridge)

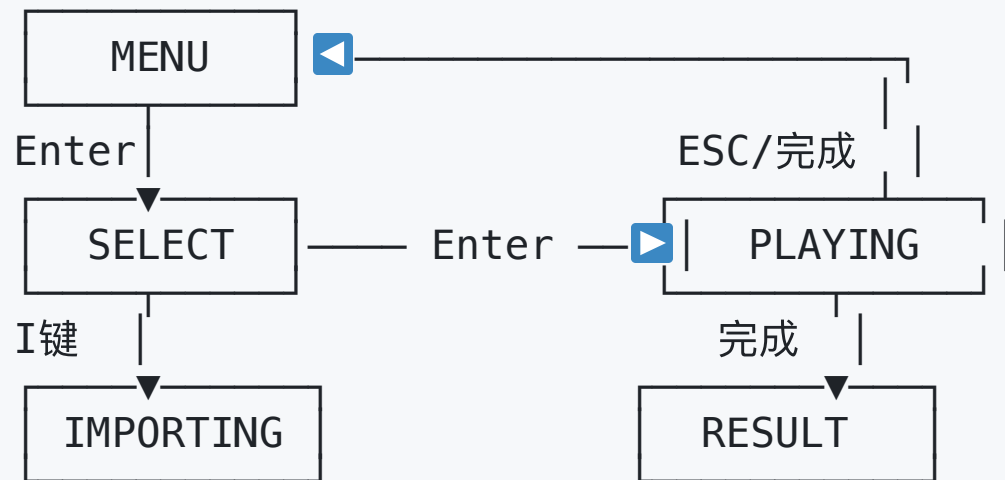
弱耦合: `Renderer` 不依赖 `Judge`, `InputManager` 不依赖 `Chart`

Loosely coupled: `Renderer` has no dependency on `Judge`, `InputManager` none on `Chart`

数据流 / Data Flow: `Input` → `Game` 协调 → `Judge` 判定 → `Chart` 更新 → `Renderer` 绘制 → `Particles` 反馈

Input → *Game orchestrates* → *Judge evaluates* → *Chart updates* → *Renderer draws* → *Particles feedback*

状态机 / State Machine



5 个状态枚举，由用户输入驱动转换

5-state enum driven by user input transitions

`autoplay_` 标志在 PLAYING 内切换手动/自动，无需额外状态

`autoplay_` flag toggles manual/auto within PLAYING — no extra state needed

创新点 ① 多级判定 / Multi-tier Judgment

判定	时间窗口	得分
PERFECT	$\leq 45\text{ms}$	350
GREAT	$\leq 90\text{ms}$	200
GOOD	$\leq 140\text{ms}$	100
MISS	$> 140\text{ms}$	0 (断 Combo)

四级时间窗口判定：越精准得分越高

Four-tier timing window: the more precise, the higher the score

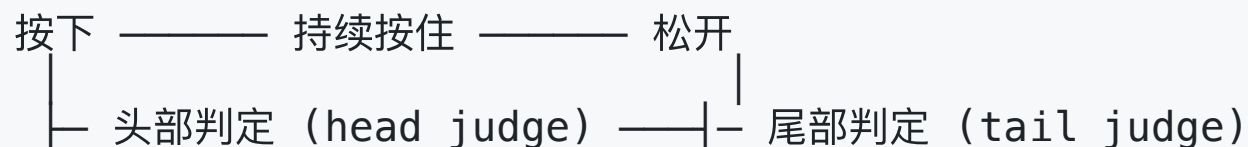
超时未按自动 MISS；空按（无对应音符）也触发 MISS

Expired notes auto-MISS; empty presses (no matching note) also trigger MISS

```
// 判定窗口常量 (Config.h)
constexpr float JW_PERFECT = 0.045f; // 45ms
constexpr float JW_GREAT  = 0.09f;  // 90ms
constexpr float JW_GOOD   = 0.14f;  // 140ms

// 核心判定逻辑 (Judge.cpp)
JudgeResult JudgeSystem::judge(float diff) const {
```

创新点 ① Hold 双段判定 / Hold Dual Judgment



头部：按键时刻 vs 音符到达时刻 → 四级判定

Head: press time vs note arrival → four-tier judgment

尾部：松开时刻 vs Hold 结束时刻 → 独立二次判定，两次得分

Tail: release time vs hold end → independent second judgment, scored twice

早松超出窗口直接 MISS，保证长按的挑战性

Early release beyond window triggers MISS, ensuring hold note challenge

```
void JudgeSystem::applyHold(JudgeResult head, JudgeResult tail) {  
    apply(head); // 头部走标准判定 (含 Combo 逻辑)  
    // 尾部额外加分，仅头部非 MISS 时有效  
    if (head != MISS && tail != MISS) {  
        counts_[idx(tail)]++; score_ += pts(tail);  
        combo_++; if (combo_ > maxCombo_) maxCombo_ = combo_;  
    }  
}
```

创新点 ② Obj-C++ 文件选择器 / Obj-C++ File Picker

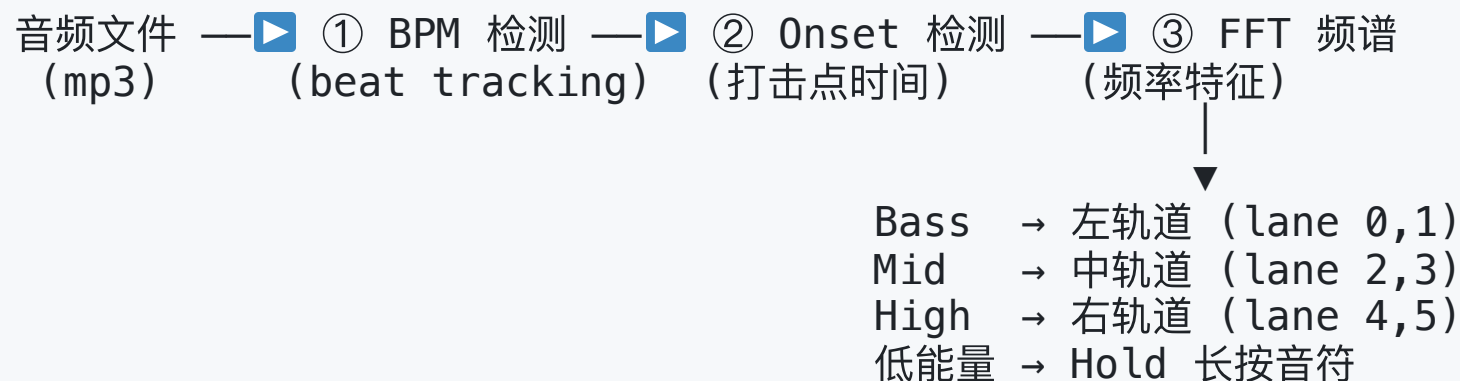
```
// FilePicker.h – C++ 接口 / C++ interface
std::string openAudioFileDialog();

// FilePicker.mm – Objective-C++ 实现 / Obj-C++ impl
#ifdef __APPLE__
#include <AppKit/AppKit.h>
std::string openAudioFileDialog() {
    @autoreleasepool {
        NSOpenPanel* panel = [NSOpenPanel openPanel];
        [panel setAllowedFileTypes:
            @[@"mp3",@"wav",@"ogg",@"flac"]];
        if ([panel runModal] == NSModalResponseOK)
            return std::string(
                [[[panel URLs] objectAtIndex:0]
                 fileSystemRepresentation]);
        return "";
    }
}
#endif
```

创新点 ③ 自动谱面生成 / Auto Chart Generation

基于 aubio 的分析

Three-stage analysis pipeline based on aubio



BPM 检测: aubio tempo → 中位数拍间距 → 换算 BPM

BPM detection: aubio tempo → median beat interval → convert to BPM

Onset 检测: 打击点时间戳 + 80ms 最小间隔

Onset detection: onset timestamps + 80ms minimum gap

FFT 频谱: spectral centroid + bass ratio → 轨道分配

FFT spectrum: spectral centroid + bass ratio → lane assignment

创新点 ③ Onset 检测 / Onset Detection

每 512 个采样点为一帧，逐帧读取音频

Read audio in frames of 512 samples each

帧能量突然增大? → 记录该时间点为 onset (打击点)
Energy spike? → Record this timestamp as an onset

过滤: 相邻 onset 间隔 < 80ms 则丢弃 (防止重复检测)

Filter: discard onsets closer than 80ms to avoid duplicates

效果: 跑完整首歌得到打击点时间列表, 即音符出现的时间

Result: a list of timestamps where notes should appear

```
// 核心去重逻辑 Core dedup logic
aubio_onset_do(od, buf, onsetOut);
if (onsetOut->data[0] != 0) {
    float t = aubio_onset_get_last_s(od);
    if (t - lastOnset > 0.08f) { // 80ms 最小间隔
        // 记录该 onset / record this onset
        lastOnset = t;
    }
}
```

创新点 ③ FFT 频谱分析 / FFT Spectrum Analysis

在每个 onset 时刻，对 512 采样点做 FFT 变换

At each onset, perform FFT on 512 samples

得到 256 个频率 bin，从中提取两个指标：

Get 256 frequency bins, extract two metrics:

bassRatio = 前 32 bin 强度 / 总强度 → 低音占比

bassRatio = first 32 bins energy / total energy → bass ratio

centroid = 加权重心位置 → 频谱质心，越高高频越多

centroid = weighted center → spectral centroid, higher = more high-freq

```
// FFT 特征提取 (ChartGenerator.cpp)
float totalEnergy = 0, weightedFreq = 0, bassEnergy = 0;
int bassBins = 256 / 8; // 前32bin = 低音区域
for (int i = 0; i < 256; i++) {
    float mag = fftC->norm[i];
    totalEnergy += mag;
    weightedFreq += mag * i;
    if (i < bassBins) bassEnergy += mag;
}
```

创新点 ③ 轨道分配 / Lane Assignment

bassRatio 大 → 低音 → 左轨道 (lane 0,1)

High bassRatio → bass → left lane (0,1)

centroid 大 → 高音 → 右轨道 (lane 4,5)

High centroid → treble → right lane (4,5)

都不突出 → 中频 → 中轨道 (lane 2,3)

Neither dominant → mid → center lane (2,3)

效果：低沉鼓点按左手，明亮高音按右手，模拟真实演奏

Low drums → left hand, bright highs → right hand, like real performance

```
if (bass > 0.5f)
    lane = (centroid < 0.15f) ? 0 : 1;           // 低音→左手
else if (centroid > 0.6f)
    lane = (centroid > 0.75f) ? 5 : 4;           // 高音→右手
else
    lane = (centroid > 0.35f) ? 3 : 2;           // 中频→中间
```

创新点 ③ 段落变化策略 / Section Variation

每 8 拍切换轨道分配模式，避免单调

Switch lane assignment mode every 8 beats to avoid monotony

模式	行为 / Behavior
0	纯频谱映射：低→左，高→右 / Pure spectrum mapping
1	频谱 ± 随机偏移 / Spectrum ± random offset
2	等差递增 / Arithmetic progression
3	频谱 ± 大幅偏移 / Spectrum ± wide offset
4	镜像： <code>5 - lane</code> / Mirror

Hold 生成条件 / Hold Generation:

`energy < 0.4 && bass > 0.3 && noteIndex % 10 == 0`

低能量持续音 + 低音特征 → Hold 长按，时长 `beatDur * (1~3)`

Low-energy sustained sounds + bass character → Hold, duration 1~3 beats

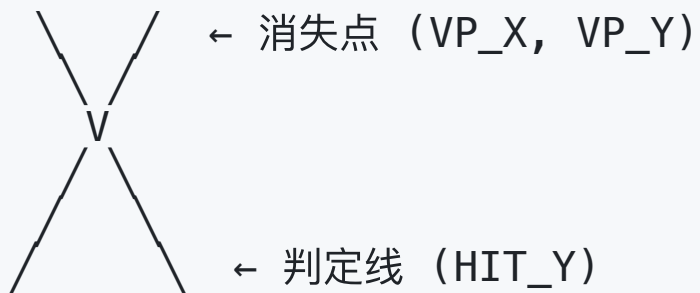
后处理 / Post-processing: 移除 Hold 区间内的 Tap 重叠 → 按时间排序 → 去重 (同lane<10ms)

Remove taps overlapping with holds → sort by time → dedup same lane

渲染系统 / Rendering System

2D 模拟 3D 透视效果

2D simulating 3D perspective effect



```
// 非线性深度映射 - 幂函数制造隧道感
float perspY(float d) {
    return VP_Y + TRACK_LEN * powf(d, 1.55f);
}
float perspX(float hx, float d) {
    return VP_X + (hx - VP_X) * d; // 线性收敛到消失点
}
// d=0 消失点 (y=15), d=1 判定线 (y=530)
```

粒子特效 / Particle Effects

常驻背景花瓣：50 个半透明圆缓慢上浮，左右正弦摆动

Background petals: 50 translucent circles float upward with sinusoidal drift

爆炸 (Explosion)：PERFECT 时 25 个粒子向四周扩散

Explosion: 25 particles burst outward on PERFECT hit

白色 + 轨道色混合，部分带 sparkle 闪光

White + lane color mix, some with sparkle effect

粒子特效 (续) / Particle Effects (cont.)

螺旋 (Spiral): PERFECT 时 8 个粒子螺旋飞出

Spiral: 8 particles fly out in spiral on PERFECT hit

角度随时间旋转, 使用相邻轨道配色

Angles rotate with time, using adjacent lane colors

火焰 (Fire): Combo > 10 时在判定线向上喷射

Fire: Upward spray above hit line when combo > 10

窄角度向上的大粒子, 营造连续击打的成就感

Narrow-angle large particles, creating a satisfying streak feel

```
// 粒子物理 (ParticleSystem.cpp)
void ParticleSystem::update(float dt) {
    for (auto& p : particles_) {
        p.pos.x += p.vel.x * dt;
        p.pos.y += p.vel.y * dt;
        p.vel.y += 400.0f * dt;    // 重力 gravity
        p.vel.x *= 0.97f;         // 水平阻力 drag
        p.life -= dt;
    }
    // erase-remove idiom: 自动回收过期粒子
```

JSON 解析器 / JSON Parser

```
struct JsonValue {  
    JsonType type;    // Null/Bool/Number/String/Array/Object  
    double num;  string str;  
    vector<JsonValue> arr;  
    vector<pair<string, JsonValue>> obj;  
    const JsonValue& operator[](const char* key) const;  
    const JsonValue& operator[](size_t i) const;  
};
```

链式访问: `root["notes"][0]["time"]`

Chain access: `root["notes"][0]["time"]`

递归下降解析 / Recursive descent: `parseValue()` → 根据首字符分发 `"` → `parseStr`, `{` → `parseObj`,
`[` → `parseArr`, 否则 → `parseNum`

约 100 行零依赖 / ~100 lines, zero dependencies

OOP Principles

封装 / Encapsulation

每个类隐藏内部实现，仅暴露必要接口

Each class hides internals, exposes only necessary interfaces

`JudgeSystem` 内部维护 score/combo, 外部只调用 `judge()` / `apply()`

`JudgeSystem` internally maintains score/combo; externally only calls `judge()` / `apply()`

单一职责 / Single Responsibility

`InputManager` 只管输入, `AudioManager` 只管音频, `Renderer` 只管渲染

`InputManager` handles input only, `AudioManager` handles audio only

组合 / Composition

`Game` 通过组合持有子系统, 无继承层级

`Game` composes subsystems, no inheritance hierarchy

设计模式 / Design Patterns

适配器 (Adapter): FilePicker.mm 把 NSOpenPanel 的 Obj-C 接口转成 C++ 函数

Adapter: FilePicker.mm wraps NSOpenPanel Obj-C interface into a C++ function

对象适配器: 内部组合 NSOpenPanel, 对外暴露 `openAudioFileDialog()`

Object adapter: composes NSOpenPanel internally, exposes `openAudioFileDialog()`

策略 (Strategy): `autoplay_` 标志切换手动/自动两种游玩行为

Strategy: `autoplay_` flag switches between manual and auto play behavior

迭代器 (Iterator): 大量使用 STL 迭代器遍历音符、粒子、歌曲列表

Iterator: STL iterators used throughout for notes, particles, song lists

C++ 基础 / C++ Basics

Makefile 构建

Makefile build

多文件项目: `.h` / `.cpp` 分离, `#pragma once` 头文件守卫

Multi-file project: `.h` / `.cpp` separation, `#pragma once` header guards

STL 容器: `vector` (音符/粒子列表)、`array<int,4>` (判定计数)

STL containers: `vector` for notes/particles, `array<int,4>` for judgment counts

文件 I/O: `std::ifstream` / `std::stringstream`

File I/O: `std::ifstream` / `std::stringstream`

C++ 特性 / C++ Features

枚举类: `enum class JudgeResult : uint8_t` — 类型安全

Enum class: type-safe, prevents implicit conversions

结构化: `for (auto& [k, v] : obj)` 遍历 JsonValue 对象

Structure for JsonValue object traversal

Obj-C++ : `.mm` 文件

Obj-C++: `.mm` files bridge C++ and macOS

程序化音频合成 / Procedural Audio Synthesis

按键音：6 个轨道独立正弦波音色 / Lane hit sounds: 6 independent sine-wave tones

```
S=440Hz(A4) D=494Hz(B4) F=523Hz(C5) J=587Hz(D5) K=659Hz(E5) L=784Hz(G5)
```

Kick 底鼓 / Kick drum:

```
buf[i] = sin(2π * (150 * e-20t + 40) * t) // 频率 190→40Hz 快速下降  
        * 0.3 * (1 - t/0.15)3 * 32767; // 模拟鼓膜振动
```

Hat 踩镲 / Hi-hat:

```
buf[i] = (rand()*2-1) * 0.1 * (1 - t/0.04)6 * 32767; // 白噪声 40ms
```

无音频文件时节拍器自动接管 / Metronome auto-fallback when no audio file

自动演示模式 / Auto-Demo Mode

选歌界面按 **A** 激活 / Press **A** in song select to activate

```
// autoplay_ 标志切换策略 (Game.cpp)
if (autoplay_) {
    // Tap: 到达前 5ms 内自动 PERFECT
    if (n.type == TAP && diff <= 0.005f && diff > -JW_PERFECT) {
        n.result = PERFECT; judge_.apply(PERFECT);
        // 触发全效果: 粒子爆炸+螺旋+火焰+屏幕闪光
        spawnExplosion(); spawnSpiral();
        if (combo > 10) spawnFire(); screenFlash_ = 1;
    }
    // Hold: 头部自动按 + 尾部自动释放
    // 自动管理 keysDown_[] 状态
}
```

Auto-release 逻辑: 释放不再需要的按键, 模拟真实演奏

Release keys no longer needed, simulates real performance

测试与验证 / Testing & Verification

20 项手动测试全部通过  / 20 manual tests all passed 

#	测试内容 / Test	状态
1-3	主菜单显示 → ENTER选歌 → UP/DOWN切换 / Menu→Select→Switch	✓
4-6	手动游玩 / 音符判定 / HOLD双段 / Manual play/Judge/Hold	✓
7-8	分数+Combo累计 / MISS断连 / Score+Combo/MISS break	✓
9-10	结算画面(S/A/B/C/D等级) / 返回主菜单 / Results→Menu	✓
11	自动演示(A键全PERFECT) / Auto-demo (all PERFECT)	✓
12	ESC退出游玩 / ESC quit during play	✓
13-14	I键导入→文件选择器→分析→游玩 / Import→Analyze→Play	✓
15	Random随机生成谱面 / Random generated chart	✓
16-17	粒子特效(爆炸/螺旋/火焰) / 屏幕闪光 / Particles/Flash	✓
18-20	节拍脉动 / 透视效果 / ESC选歌返回 / Pulse/Persp/ESC	✓

总结 / Summary

核心创新 / Core Innovations:

多级下落判定系统、Obj-C++ 原生文件选择器、音频分析自动谱面生成

Multi-tier judgment, native Obj-C++ file picker, audio-driven auto chart generation

全栈技术 / Full-stack tech: C++17 · raylib · aubio · Obj-C++ · AppKit · Makefile

代码规模 / Code scale: 14 headers + 11 sources $\approx$ 2500 lines · 手写 JSON parser

设计亮点 / Design highlights: 弱耦合组合式架构 · 程序化音频合成 · 3D透视渲染 · 粒子特效系统

谢谢! Q & A

aeacrA

按键 / Keys: S D F J K L | 选歌界面按 A 自动演示 / Press A for auto-demo